DX Cluster Aggregator

for macOS

User Manual

Version 1.8.9

Aggregate FT8/FT4 spots from multiple WSJT-X/JTDX instances
and DX Cluster nodes into a unified telnet cluster server

Original Windows Application
Vinod VU3ESV / LB9KJ

macOS Version Conceptualised by
Manoj VU2CPL

macOS Version Developed with
Claude Code (Anthropic)

Table of Contents

1. Introduction

2. System Requirements

3. Installation

- 3.1 From Pre-built App Bundle
- 3.2 Building from Source
- 3.3 Creating the .app Bundle
- 3.4 Updates

4. Getting Started

- 4.1 Main Window Overview
- 4.2 Setting Your Callsign

5. Configuring UDP Sources (WSJT-X / JTDX)

- 5.1 Adding a UDP Source
- 5.2 Multiple Instances

6. Configuring DX Cluster Sources

- 6.1 Adding a DX Cluster Node
- 6.2 Authentication

7. Broadcast Destinations

- 7.1 UDP Broadcast
- 7.2 TCP Telnet Cluster Server

8. Monitoring & Spot Aggregation

- 8.1 Starting Monitoring
- 8.2 CQ-Only Filter
- 8.3 New-Only Filter
- 8.4 Understanding the Spots Table

9. ClubLog Integration & DX Alerts

- 9.1 Required Credentials
- 9.2 Configuration and Refresh
- 9.3 Band Filter for Import
- 9.4 Include Unconfirmed
- 9.5 Alert Types and Highlighting

10. Notifications (Telegram & macOS)

- 10.1 Telegram Setup
- 10.2 macOS Notifications
- 10.3 Cooldown
- 10.4 Notify-On Selection

11. Connecting Logging Software

12. Troubleshooting

13. What's New in 1.7.0

14. Credits & Acknowledgements

1. Introduction

DXClusterAggregator is a macOS application designed for amateur radio operators. It collects FT8, FT4, and other digital mode spots from multiple sources and aggregates them into a unified DX cluster feed that can be consumed by logging software such as Logger32, DXLab, N1MM+, Log4OM, and others.

The application supports two types of spot sources:

- **UDP Sources** - Direct connections to WSJT-X, JTDX, or other software that broadcasts decoded FT8/FT4 spots via the WSJT-X UDP protocol
- **DX Cluster Nodes** - Telnet connections to traditional DX cluster servers (DX Spider, AR-Cluster, CC Cluster, etc.) to receive spots from the worldwide cluster network

All spots from all sources are combined and rebroadcast through a built-in telnet cluster server and/or UDP, allowing your logging software to receive a single aggregated stream of spots from multiple radios, SDRs, and cluster nodes.

This macOS version is a native Swift/SwiftUI port of the original Windows application created by Vinod VU3ESV/LB9KJ. The macOS version was conceptualised by Manoj VU2CPL.

2. System Requirements

Requirement	Details
Operating System	macOS 14.0 (Sonoma) or later
Architecture	Apple Silicon (M1/M2/M3/M4) and Intel
Disk Space	Less than 5 MB
Network	Local network access for UDP/TCP
Software	WSJT-X, JTDX, or compatible decoder (optional)

3. Installation

3.1 From Pre-built App Bundle

- Locate **DXClusterAggregator.app** in the distribution folder
- Drag it to your **/Applications** folder (or run from any location)
- Release builds are signed and notarised with an Apple Developer ID (hardened runtime), so they open normally on a double-click — no Gatekeeper workaround needed
- **Fallback only** (e.g. an unstapled build, or a copy built from source): go to **System Settings > Privacy & Security** and click **"Open Anyway"**

If Gatekeeper still blocks a self-built copy, open Terminal and run the following to clear the quarantine attribute:

```
xattr -cr /path/to/DXClusterAggregator.app
```

Then right-click the app and select "Open" for the first launch. After that it will open normally with a double-click.

3.2 Building from Source

If you have the source code and Xcode Command Line Tools installed (install with: xcode-select --install):

Step 1: Clone and build

```
git clone https://github.com/vu2cpl/DXClusterAggregator-macOS.git
cd DXClusterAggregator-macOS
swift build -c release
```

3.3 Creating the .app Bundle

After building from source, you can create a proper macOS .app bundle that can be launched from Finder, placed in the Dock, or copied to /Applications:

Step 1: Create the bundle directory structure

```
mkdir -p DXClusterAggregator.app/Contents/MacOS
mkdir -p DXClusterAggregator.app/Contents/Resources
```

Step 2: Copy the binary and icon

```
cp .build/release/DXClusterAggregator DXClusterAggregator.app/Contents/MacOS/
cp AppIcon.icns DXClusterAggregator.app/Contents/Resources/
echo -n "APPL????" > DXClusterAggregator.app/Contents/PkgInfo
```

Step 3: Create the Info.plist

Create the file DXClusterAggregator.app/Contents/Info.plist with the following content (a template is provided in the project README):

Key	Value
CFBundleName	DXClusterAggregator
CFBundleDisplayName	DX Cluster Aggregator
CFBundleIdentifier	com.vu2cpl.dxclusteraggregator
CFBundleVersion	1.8.9
CFBundleExecutable	DXClusterAggregator
CFBundlePackageType	APPL
CFBundleIconFile	AppIcon
LSMinimumSystemVersion	14.0
NSHighResolutionCapable	true

The full Info.plist XML template is available in the project README on GitHub.

Step 4: Sign and launch

```
codesign --force --deep --sign - DXClusterAggregator.app
```

```
open DXClusterAggregator.app
```

Step 5 (Optional): Install to Applications

```
cp -r DXClusterAggregator.app /Applications/
```

> *If sharing the built .app with others, they will need to run xattr -cr /path/to/DXClusterAggregator.app and right-click > Open on the first launch, as the app is not notarised through the Apple Developer Program.*

3.4 Updates

About 10 seconds after launch, and then once a day for as long as it keeps running, the app asks GitHub whether a newer release exists. If one does, it shows the new version and its release notes:

- **Download** opens the release page in your browser. Nothing is downloaded or installed automatically
- **Skip This Version** keeps the automatic check quiet about that release
- **Remind Me Later** asks again at the next daily check

Untick **Check for updates automatically** in the settings panel to turn it off. **Check for Updates...** in the app menu (under About) checks right away and always reports: the update, "You're up to date", or why the check failed.

> *The only request is an anonymous GET of api.github.com/repos/vu2cpl/DXClusterAggregator-macOS/releases/latest: no account or token, and nothing is sent beyond the app's name and version. An automatic check that fails (offline, GitHub busy) stays silent and does not count towards the day: it is tried again about an hour later, or at the next launch. Development builds (a version containing "dev") never check on their own.*

4. Getting Started

4.1 Main Window Overview

When you launch DXClusterAggregator, you will see the main window with the following sections from top to bottom. The **Hide Settings** button in the header collapses the entire configuration panel (Configuration, UDP Sources, DX Cluster, ClubLog) so the spots table can use the full window. Click **Show Settings** to bring the configuration back.

Section	Purpose
Header	App name and version number
Configuration	Callsign, TCP cluster port, spot log cap, update check, broadcast destinations
UDP Sources	List of WSJT-X/JTDX UDP sources with IP, port, and status
DX Cluster Nodes	List of telnet DX cluster sources with address, credentials, and status
Controls	CQ-Only filter, Minimize on Start, Clear Spots, Start/Stop Monitoring
Spots Table	Real-time display of aggregated spots from all sources
Status Bar	Connection status, active source count, and total spot count

4.2 Setting Your Callsign

Enter your callsign in the **Callsign** field at the top of the window. This callsign is used as the spotter identifier when rebroadcasting spots through the cluster server. The callsign is automatically converted to uppercase. It is also used as the default username when adding new DX Cluster sources.

5. Configuring UDP Sources (WSJT-X / JTDX)

UDP Sources receive decoded FT8/FT4/JT65/JT9 spots directly from WSJT-X, JTDX, or any software that implements the WSJT-X UDP protocol. The app parses the binary WSJT-X protocol messages (Status and Decode types) to extract spot information.

5.1 Adding a UDP Source

Click **"Add UDP Source"** in the UDP Sources section. Configure the following fields:

Field	Description	Default
Name	A label to identify this source (e.g., "WSJT-X 20m")	Source N
Listen IP	IP to listen on. Use 0.0.0.0 for all interfaces	0.0.0.0
Port	UDP port number matching your WSJT-X/JTDX setting	2237

Enabled	Toggle to enable/disable this source	On

Note: In WSJT-X, go to File > Settings > Reporting and set the UDP Server address to 127.0.0.1 (or your Mac's IP if running on another machine) and the port to match the port configured here.

5.2 Multiple Instances

You can add multiple UDP sources to aggregate spots from several WSJT-X/JTDX instances running simultaneously. Each instance must be configured to broadcast on a different UDP port. For example:

- WSJT-X on 20m band: UDP port 2237
- JTDX on 40m band: UDP port 2238
- WSJT-X on 15m band: UDP port 2239

6. Configuring DX Cluster Sources

DX Cluster sources connect to traditional DX cluster telnet servers to receive spots from the worldwide amateur radio spotting network. These spots are aggregated alongside your local WSJT-X/JTDX decodes.

6.1 Adding a DX Cluster Node

Click **"Add DX Cluster"** in the DX Cluster Nodes section. Configure the following fields:

Field	Description	Default
Name	A label for this cluster (e.g., "VE7CC")	Cluster N
Address	Hostname or IP of the cluster node	(empty)
Port	Telnet port number	7300
Username	Your callsign for login	Your callsign
Password	Password if required (many clusters do not require one)	(empty)
Enabled	Toggle to enable/disable	On

Popular DX Cluster Nodes

Node	Address	Port
VE7CC	dxc.ve7cc.net	23
W9PA	dxc.w9pa.net	7300
GB7DXC	gb7dxc.dxcluster.co.uk	7300
WA9PIE	dxc.wa9pie.net	7300
K1TTT	k1ttt.net	7300

6.2 Authentication

The application automatically detects login and password prompts from the cluster server. Most DX cluster nodes only require a callsign (no password). The app sends your username when it detects a login prompt, and your password (if provided) when it detects a password prompt. Connection status is shown next to each cluster entry as a coloured status pill. The pill is also a button: clicking it drops the session and redials immediately - useful to kick a misbehaving cluster without stopping monitoring. Pill states:

- **Green** - proven live: the cluster acknowledged the login, or spots are arriving (which also covers clusters that need no login). The pill shows the session spot count and how long ago the last spot arrived (e.g. "128 · 9 sec")
- **Yellow, "no spots"** - TCP is up and the login has been sent, but the session is not yet proven: no login acknowledgement and no spots so far. A cluster whose login processing is down can sit in this state - the watchdog recycles it after 2 minutes
- **Orange, "connecting" / "retry Ns"** - down: the connection is being established, or it dropped (or the watchdog recycled a dead session) and a retry is scheduled with increasing backoff. Sessions that go silent for 15 minutes while live are also recycled automatically

7. Broadcast Destinations

Aggregated spots are rebroadcast through two mechanisms:

7.1 UDP Broadcast

Two UDP broadcast destinations can be configured. Each has an IP address and port. Spots are sent as DX cluster-formatted text lines via UDP. Click **Save** after changing the settings. These can be used to feed spots to other applications on the local machine or across the network.

7.2 TCP Telnet Cluster Server

The app runs a built-in TCP telnet cluster server. Any logging software that supports connecting to a DX cluster via telnet can connect to this server to receive the aggregated spot feed. The default port is **7575**.

Configure your logging software to connect to:

- **Host:** 127.0.0.1 (if on the same machine) or your Mac's IP address
- **Port:** 7575 (or whatever you configured)

The status bar shows the number of connected telnet clients.

8. Monitoring & Spot Aggregation

8.1 Starting Monitoring

Click the green **"Start Monitoring"** button to begin. This will:

- Start a UDP listener for each enabled UDP source
- Connect to each enabled DX Cluster node
- Start the TCP telnet cluster server
- Configure the UDP broadcast destinations

> *While monitoring is active, source settings cannot be modified. Click the red **"Stop Monitoring"** button to stop and reconfigure.*

8.2 CQ-Only Filter

Enable the **CQ Only** toggle to filter spots so that only CQ calls are displayed and rebroadcast. This reduces the volume of spots to only stations calling CQ, which are the ones available to work. QSO exchanges (reports, RRR, 73) are filtered out.

8.3 New-Only Filter

Enable the **New Only** toggle to filter spots so that only those matching an enabled ClubLog alert are displayed and rebroadcast. A spot passes the filter if its classification is New DXCC, New Slot, New Band, or New Mode (whichever you have enabled in the ClubLog section). Worked stations and unclassified spots are hidden. This filter requires ClubLog data to be loaded via Refresh - without log data, every spot is unclassified and nothing will be shown.

> *Tip: combine **CQ Only** + **New Only** for the cleanest possible feed - only stations calling CQ that you actually need.*

8.4 Understanding the Spots Table

The spots table displays all aggregated spots in real-time:

Column	Description
Time	UTC time of the spot (HHmm format)
Source	Name of the source that provided this spot
Callsign	The DX station callsign extracted from the message
Freq (MHz)	Frequency in MHz (dial frequency + audio offset)
SNR	Signal-to-noise ratio in dB (from WSJT-X decodes)
Mode	Operating mode (FT8, FT4, CW, SSB, etc.)
Message	Full decoded message or spot comment

> *CQ spots are highlighted with a green background in the table for easy identification.*

9. ClubLog Integration & DX Alerts

The ClubLog integration (new in v1.3.0) downloads your personal log from ClubLog and classifies each incoming spot as new DXCC, new slot, new band, new mode, or already worked. Spots are color-highlighted in the spots table so you can instantly see which stations are worth working.

9.1 Required Credentials

You will need the following from your ClubLog account (free at https://clublog.org). Only the first three are yours to supply:

Field	What it is	Where to get it
Callsign	Your amateur radio callsign	Your ClubLog account callsign
Email	Email address registered with ClubLog	Your ClubLog account email
App Password	API token - NOT your login password	ClubLog > Settings > App Passwords
API Key	Optional - the app has one built in	ClubLog section > Advanced

*The **API Key** is the one field you can ignore, which is why it sits out of the way under **Advanced** at the foot of the ClubLog section. It identifies the application rather than the operator, and a released DXClusterAggregator ships with a working developer key for the country-file download - so leave it blank and Refresh will just work. Fill it in only if you would rather the download counted against a key of your own, or if you built the app from source (those builds carry no key, and the Advanced section opens by default to say so); clearing the field goes back to the built-in key.*

All credentials are stored locally on your Mac in UserDefaults. They are not transmitted anywhere except to ClubLog's own servers when you click Refresh.

9.2 Configuration and Refresh

In the ClubLog Integration section:

- Enter your **Callsign**, **Email** and **App Password** - that is everything a released build needs (the API Key lives under **Advanced** and can stay empty)
- Select which **Alert Types** you want highlighted (New DXCC, New Slot, New Band, New Mode)
- Click **Refresh from ClubLog** - this downloads the country file and your full ADIF log
- Wait for the status message to show the QSO and DXCC count (depending on log size, this may take 30-120 seconds)

The downloaded data is cached locally at ~/Library/Application Support/DXClusterAggregator/ so the app loads instantly on next launch without re-downloading. You should refresh periodically (e.g. after each logging session) to keep the worked-status data up to date.

9.3 Band Filter for Import

The **Import Bands** grid in the ClubLog section lets you choose which bands' QSOs are imported from your ClubLog log. Useful if you only operate certain bands and want a smaller, faster matrix.

- **All** button: import every band (default)
- **HF Only** button: 160M through 10M

- Individual band checkboxes: pick exactly the bands you want

Changes take effect on the next Refresh (the import filter is applied during ADIF parsing).

9.4 Include Unconfirmed

Enable the **Inc. Unconfirmed** toggle to make worked-but-unconfirmed slots count as still needed. A slot is considered confirmed if any of LOTW_QSL_RCVD, QSL_RCVD, or EQSL_QSL_RCVD is set to Y in the ADIF record (also recognises ClubLog's matched flag). When this toggle is ON, the alert classifier compares against your confirmed slots only, so QSOs awaiting confirmation will trigger New DXCC / New Slot / etc. alerts. This is useful for award-chasers who want to re-work stations until confirmation is received.

9.5 Alert Types and Highlighting

A **slot** means a specific DXCC + Band + Mode combination. For example, if you have worked Japan on 20M FT8 and 40M CW, then a spot of a Japanese station on 15M FT8 is a **new slot** - Japan is worked (so not new DXCC), 15M is worked (for other entities), FT8 is worked, but this specific JA+15M+FT8 combination is not. Slots matter for awards like 5BDXCC, 9BDXCC, and triple-play.

Indicator	Level	Meaning
■ red row	New DXCC	You have never worked this DXCC entity
■ orange row	New Slot	You have worked this DXCC but not on this exact band+mode combination
■ blue row	New Band	You have worked this DXCC but not on this band (any mode)
■ amber row	New Mode	You have worked this DXCC but not in this mode (any band)
■ no highlight	Worked	You have already worked this station/slot - not needed
(blank)	Unknown	Classification not possible (DXCC not resolved, or no log data loaded)

Priority order: New DXCC beats New Slot beats New Band beats New Mode beats Worked. If you disable a specific alert type via the toggle, spots at that level are displayed as Worked (no highlight).

The spots table now includes two extra columns: **DXCC** (the entity name, e.g. "UNITED STATES") and **Band** (e.g. "20M"). A small colored indicator appears at the far-left of each row showing the alert level at a glance.

10. Notifications (Telegram & macOS)

DXClusterAggregator can push alerts to your Telegram chat and/or to the macOS Notification Center whenever a spot matching your selected alert types arrives. Alerts are throttled per callsign by a configurable cooldown to prevent spam.

10.1 Telegram Setup

- Create a Telegram bot via @BotFather and copy the **Bot Token**
- Send /start to your bot, then look up your **Chat ID** via @userinfobot or by visiting https://api.telegram.org/bot<TOKEN>/getUpdates
- In DXClusterAggregator, open the **Notifications** section and enable **Telegram**
- Paste the Bot Token and Chat ID, then click **Send Test**

10.2 macOS Notifications

Enable the **macOS Notifications** toggle. The first time you enable it, macOS will prompt you to allow notifications from DXClusterAggregator. Click **Allow**. Banners will then appear in the corner of your screen for each matching spot.

> *If you accidentally denied permission, open System Settings > Notifications, find DXClusterAggregator in the list, and re-enable Allow Notifications.*

10.3 Cooldown

Set the **Cooldown** to a value between 5 and 60 minutes (default 15). Within this window, repeat spots of the same callsign do not trigger another notification. This prevents getting flooded by FT8 stations that decode every 15 seconds.

10.4 Notify-On Selection

Use the **Notify on** checkboxes to choose which alert levels should trigger a notification - independent of the table-highlight toggles in the ClubLog section. For example, you might highlight all four levels visually but only get notifications for new DXCC and new slot.

11. Connecting Logging Software

DXClusterAggregator acts as a telnet DX cluster server. To connect your logging software:

Logger32

- Go to Setup > Telnet > DX Cluster
- Set Host to **127.0.0.1** and Port to **7575**
- Click Connect

N1MM+

- Go to Window > Telnet
- Enter **127.0.0.1:7575** as the cluster address

Log4OM

- Go to Settings > Cluster
- Add a new cluster with address **127.0.0.1** port **7575**

Any Telnet Client

You can also test the cluster server using a terminal:

```
telnet 127.0.0.1 7575
```

12. Troubleshooting

No spots appearing from WSJT-X

Verify the UDP port in WSJT-X Settings > Reporting matches the port configured in DXClusterAggregator. Ensure WSJT-X is set to broadcast to 127.0.0.1 (or your Mac's IP). Check that the source is enabled and shows "Active" status.

DX Cluster not connecting

Verify the cluster address and port are correct. Check your internet connection. Some clusters may be down - try a different node. Check if a firewall is blocking outbound TCP connections.

DX Cluster shows "Waiting..."

The cluster node may be unreachable or the DNS lookup failed. Try using an IP address instead of a hostname.

macOS security warning on launch

Official notarised releases open normally. This warning only appears for self-built (ad-hoc signed) copies - go to System Settings > Privacy & Security and click "Open Anyway", or run: xattr -cr /path/to/DXClusterAggregator.app

Logging software cannot connect

Ensure monitoring is started (green "Start Monitoring" button). Verify the TCP cluster port (default 7575) is not blocked by a firewall. Check that the port is not already in use by another application.

Port already in use

Another application is using the same port. Either close that application or change the port in DXClusterAggregator to an unused port number.

Spots from cluster but no frequency

DX Cluster spots include frequency information from the cluster server. This is normal - the frequency comes from the spotter, not from your radio.

13. What's New in 1.7.0

LoTW User Marker

A new green dot appears after callsigns in the Callsign column for stations known to use ARRL Logbook of The World. The LoTW user list is downloaded from ARRL (default URL https://lotw.arrl.org/lotw-user-activity.csv) via the **Refresh LoTW users** button in the ClubLog section. The list is cached locally and reused across app launches. Toggle **Mark LoTW users** to show or hide the marker without losing the database.

Beacon Detection

Known beacons (all 18 NCDXF/IBP rotating beacons plus common national beacons) are labelled in the DXCC column with their location and network, and marked with a ■ icon at the left of the row instead of an alert colour. Callsigns carrying a **/B** or **/BCN** suffix are also treated as beacons even when not in the explicit database. Beacons never trigger alerts or notifications.

Digital Modes Grouped as DATA

FT8, FT4, RTTY, JT65, JT9, MSK144, PSK and every other digital mode now share one 'DATA' slot for DXCC tracking purposes - matching how DXCC / LOTW / ClubLog awards count them. CW and voice modes (SSB / AM / FM / DIGITALVOICE etc.) remain separate. An FT4 spot no longer triggers a 'new mode' alert just because your log only has FT8 for that entity.

Time-Bounded ClubLog Rules

The ClubLog cty.xml contains historical prefix rules with <start> and <end> dates (e.g. the KARELO-FINN REPUBLIC prefix ended 1960-06-30). Those are now filtered by the current date, so modern Russian calls like RN1TV correctly resolve to EUROPEAN RUSSIA instead of the deleted entity.

Auto Start on Launch

New **Auto Start** toggle in the controls row. When enabled, monitoring kicks off automatically shortly after the app launches. Combined with **Hide on Start** this effectively runs the aggregator as a menu-bar background service from the moment you log in.

DX Cluster Auto-Reconnect

When a DX cluster connection drops (network blip, remote server restart, etc.) the client now retries automatically with capped exponential backoff: 10s, 30s, 60s, 120s, then 5 minutes repeating. The Status column shows 'Reconnect in Xs (try N)' during the wait. A successful reconnect resets the counter. Clicking Stop Monitoring, disabling the source, or removing it stops the retry loop.

Auto-Clear + Spot Log File

The controls row has an **Auto Clear** field (0-120 minutes, default 60). A background timer prunes spots older than the cutoff. Before deletion the pruned spots are appended to ~/Library/Application Support/DXClusterAggregator/DXC Spots.txt (tab-separated with a header). The manual **Clear** button also

writes to the same file. Setting auto-clear to 0 disables the prune.

The log file is size-capped via the **Spot Log Cap** field in the configuration row (default 100 MB, range 0-1024). When an append pushes the file past the cap, the oldest entries are trimmed away automatically (the file is cut back to about 75% of the cap, so the trim only runs occasionally). Set the cap to 0 to keep every spot forever - note the file then grows without bound (hundreds of MB over a few months of continuous monitoring).

Hide Duplicates Filter

New toggle next to CQ Only / New Only. When enabled, repeat spots of the same call / band / mode within a 60-second window are collapsed to a single row. Toggle off to see every decode.

Sortable + Resizable Spots Table

The spots list is now a native macOS Table: drag between column headers to resize any column, and click a header to sort by it (click again to toggle ascending/descending). Default order is Time, newest first.

Band + Source Display Filters

Two new dropdown menus in the controls row: **Sources** (pick which UDP and DX Cluster sources to display) and **Bands** (pick which bands to watch). Both are multi-select with All / preset / individual options. Independent from the ClubLog Import Bands setting - import all bands to the matrix, then narrow the live feed to whatever you're actively watching. Notifications also honour these filters.

Universal Binary

The shipped .app is now a universal binary (arm64 + x86_64), running natively on both Apple Silicon and Intel Macs.

13.x Earlier Releases

Versions 1.0-1.6 added the WSJT-X UDP listener, DX cluster telnet client, built-in telnet cluster server, dual UDP broadcast, ClubLog integration with alert classification, Telegram and macOS notifications with per-callsign cooldown, and the project rename from FT8ClusterAggregator to DXClusterAggregator.

14. Credits & Acknowledgements

Original Windows Application

The original DXClusterAggregator was developed as a Windows .NET application by **Vinod, VU3ESV / LB9KJ**. His vision of aggregating FT8 spots from multiple sources into a unified DX cluster feed laid the foundation for this project.

---

macOS Version

The macOS version of DXClusterAggregator was conceptualised by **Manoj, VU2CPL**. This native Swift/SwiftUI port brings the functionality of the original Windows application to the Mac platform, with enhancements including multiple UDP source support, DX Cluster telnet node integration, and a modern macOS user interface.

Development

The macOS application was developed with the assistance of **Claude Code** by Anthropic, an AI-powered software engineering tool.

Open Source Libraries & Protocols

- **WSJT-X UDP Protocol** - Joe Taylor K1JT and the WSJT-X development team
- **DX Cluster Protocol** - DX Spider and AR-Cluster communities
- **Apple Network Framework** - For TCP/UDP networking on macOS
- **SwiftUI** - Apple's modern UI framework for macOS

---

73 de VU2CPL

April 2026